Comparación de FCFS y Round Robin

Informe técnico del proyecto de Sistemas Operativos · Opción 1 · Actualizado el 3 de noviembre de 2026

Integrantes: Esteban Peña y Andrés Gámez

El simulador compara dos políticas de planificación de CPU y registra tiempos, estados y tramos de ejecución. Round Robin mejora la respuesta inicial en cuatro de los cinco escenarios; en E5 ambas políticas empatan. Su efecto sobre espera y retorno depende de la carga. Las cifras proceden de ejecuciones reproducibles incluidas en la carpeta evidencias.

Problema y objetivo

Cuando varios procesos requieren una CPU, el orden de atención afecta cuánto esperan y cuándo reciben servicio. El objetivo es implementar FCFS y Round Robin, compararlos bajo las mismas entradas y explicar sus efectos mediante métricas, trazas y pruebas controladas.

Fundamentación

FCFS atiende la cola por orden de llegada y mantiene la CPU hasta finalizar o bloquearse. Round Robin limita cada turno mediante un quantum y reinserta al final al proceso que aún necesita CPU. Una tarea larga al frente de FCFS puede retrasar tareas cortas, comportamiento conocido como efecto convoy [1].

El modelo diferencia procesos listos, en ejecución, bloqueados y finalizados; un proceso bloqueado por E/S vuelve a listo cuando termina su espera [2]. La creación se representa mediante objetos de Python y no mediante procesos del sistema operativo anfitrión.

Diseño de la solución

models.py representa y valida los procesos. algorithms.py usa una cola FIFO de listos y una cola de eventos ordenada por instante. FCFS y RR comparten el motor de simulación, con un límite de turno para RR. scenarios.py define las entradas y main.py ejecuta las comparaciones y exporta CSV y JSON. test_simulator.py contiene la validación automatizada.

Cada algoritmo trabaja sobre copias nuevas de la entrada. Esto impide que la CPU ya consumida por una ejecución contamine la siguiente. Un proceso terminado conserva CPU restante cero y sus métricas finales.

Alcance

Se modela una CPU, tiempos enteros y E/S de duración fija sin cola de dispositivo. La CPU y las E/S pueden avanzar en paralelo en el tiempo simulado. No se mide la velocidad real de Windows, Linux ni del procesador del equipo.

Escenarios y reglas de ejecución

E1, E2 y E3 conservan exactamente los procesos y quantum del ZIP original. E4 y E5 se incorporaron durante la revisión para comprobar bloqueo y ocio. Cada política recibe una carga idéntica. Los tiempos se expresan en unidades simuladas.

Caso	Procesos como PID llegada CPU	Q de RR	Propósito
E1	P1 (0,5); P2 (1,3); P3 (2,8); P4 (3,6)	3	Llegadas escalonadas
E2	P1 (0,12); P2 (1,2); P3 (2,4)	3	Efecto convoy
E3	P1 (0,4); P2 (0,6); P3 (0,8)	4	Llegadas simultáneas
E4	P1 (0,6); P2 (1,5); P3 (2,3). P1: E/S de 4 tras consumir 2 de CPU	2	Bloqueo y reactivación
E5	P1 (3,2); P2 (10,3)	2	Ocio inicial e intermedio

Admisión y desempates

Al comenzar se crean los registros NUEVO; en su instante de llegada pasan a LISTO. Las llegadas y finalizaciones de E/S se incorporan por tiempo y orden de registro. A igual instante, las llegadas originales preceden a los desbloqueos. Los eventos disponibles al final del turno se admiten antes de reinsertar al proceso que agotó quantum. Las llegadas no expropian una CPU ocupada.

Definición de métricas

Métrica	Definición operativa
Retorno y respuesta	Retorno = finalización - llegada. Respuesta = primer inicio de CPU - llegada [1].
Espera en listo	Espera = retorno - CPU total - tiempo bloqueado. También se acumula directamente por cada permanencia en LISTO.
Promedios y máximos	Promedio aritmético por proceso; máximos de espera y respuesta como señales de demoras individuales.
Cambios y despachos	Cambio: tramos contiguos de procesos distintos. Excluye primer despacho y pasos desde/hacia ocio. Despacho: cada asignación de CPU.
Utilización y productividad	CPU ocupada / tiempo hasta el último fin, desde t=0; productividad = procesos terminados / ese mismo intervalo.

El costo temporal de despacho y cambio de contexto se fija en cero. Los conteos no representan un tiempo medido. Para entrada vacía se devuelve una lista vacía y métricas cero por convención de la interfaz.

Resultados e interpretación

Fuente: evidencias/resumen.csv. Espera, retorno y respuesta son medias. CC identifica los cambios de contexto según la definición anterior. Se muestran dos decimales; los CSV conservan mayor precisión.

Caso	Política	Espera	Retorno	Respuesta	CC	CPU %
E1	FCFS	5.75	11.25	5.75	3	100.00
E1	RR Q=3	8.50	14.00	3.00	7	100.00
E2	FCFS	7.67	13.67	7.67	2	100.00
E2	RR Q=3	4.67	10.67	1.67	5	100.00
E3	FCFS	4.67	10.67	4.67	2	100.00
E3	RR Q=4	6.00	12.00	4.00	4	100.00
E4	FCFS	3.33	9.33	2.00	3	100.00
E4	RR Q=2	5.33	11.33	1.00	7	100.00
E5	FCFS	0.00	2.50	0.00	0	38.46
E5	RR Q=2	0.00	2.50	0.00	0	38.46

E1: RR baja la respuesta de 5.75 a 3.00, pero aumenta la espera de 5.75 a 8.50 y el retorno de 11.25 a 14.00. Repartir turnos favorece el primer acceso y retrasa algunas finalizaciones. E2: RR reduce la espera de 7.67 a 4.67 y el retorno de 13.67 a 10.67 al permitir que las tareas cortas avancen antes de terminar la larga.

E3: RR mejora ligeramente la respuesta (4.67 a 4.00), mientras el retorno aumenta de 10.67 a 12.00. E4: el bloqueo de P1 libera la CPU; la espera media excluye las cuatro unidades de E/S. E5: ambos algoritmos tienen espera cero y utilizan la CPU 5 de 13 unidades, es decir, 38.46 %. Los huecos explican la baja utilización.

Sensibilidad al quantum en E2

Quantum	Espera media	Retorno medio	Respuesta media	Cambios
1	4.33	10.33	0.33	11
3	4.67	10.67	1.67	5
6	5.67	11.67	3.67	3
12	7.67	13.67	7.67	2

En esta carga, Q=1 da la menor respuesta y 11 cambios; Q=12 reproduce los tiempos de FCFS y da 2 cambios. No se puede concluir que Q=1 sea óptimo para un sistema real, porque aquí no se descuenta CPU útil por los cambios. Ninguna política resulta superior en todas las métricas y escenarios.

Evidencias y validación

La ejecución fue realizada con Python 3.12.14 en Windows. El simulador y sus pruebas utilizan únicamente la biblioteca estándar. entorno.json registra la plataforma, el comando y SHA-256 de los archivos ejecutados. entradas.json conserva las cargas exactas.

Evidencia	Qué permite comprobar
procesos.csv	Llegada, CPU, bloqueo, primer inicio, fin y métricas por proceso.
cpu.csv y estados.csv	Tramos de CPU y ocio; instante, motivo y estado de cada transición.
ejecucion.txt y pruebas.txt	Salida real de los cinco escenarios y de unittest.
compatibilidad_original.json	Igualdad exacta por proceso en las seis ejecuciones de E1 a E3, antes y después de corregir.
validacion_adicional.txt	Recálculo de CSV y manejo de un destino de salida que era un archivo.

Pruebas realizadas

Se aprobaron 19 métodos de prueba. Se incluyeron resultados manuales, llegadas empatadas y en el límite del quantum, CPU ociosa, uno o varios bloqueos, entrada vacía, validaciones de errores y reutilización de procesos. Se verificaron CPU restante cero, finalización, conservación de CPU y la identidad entre retorno, espera, CPU y bloqueo.

Un método genera 200 cargas reproducibles con semilla 2620. Cada una se compara en FCFS y RR con Q=1, Q=2 y Q=5 frente a un segundo modelo que avanza por ticks y no llama al motor del simulador: 800 comparaciones de inicio, fin, espera y bloqueo. Compartir las mismas reglas de desempate es necesario; por eso también hay casos calculados a mano.

Ejemplo calculado manualmente

Dos procesos: P1 llega en 0 y requiere 5 de CPU; P2 llega en 1 y requiere 3. Con RR y Q=2, la traza es 0-2 P1, 2-4 P2, 4-6 P1, 6-7 P2 y 7-8 P1. P1 termina en 8: retorno 8, espera 3, respuesta 0. P2 termina en 7: retorno 6, espera 3, respuesta 1. Esta secuencia está comprobada por test_rr_manual.

Confirmación del bloqueo

En E4, P1 ejecuta de 0 a 2 y queda BLOQUEADO hasta 6. El evento de fin de E/S lo devuelve a LISTO en 6; volver a listo no garantiza CPU inmediata. El intervalo bloqueado es 4 en ambas políticas. estados.csv permite reconstruir el recorrido sin depender de capturas de pantalla.

Las pruebas aportan evidencia para las cargas verificadas, no una demostración formal para toda entrada posible. Al ser simulaciones deterministas, repetir una carga produce los mismos datos; no se requieren promedios estadísticos de ejecuciones de un SO real.

Equidad riesgos y límites

Equidad e inanición

En E1 el máximo de respuesta pasa de 13 en FCFS a 6 en RR; en E2, de 12 a 3; en E3, de 10 a 8. Es evidencia de mejor acceso inicial en estas cargas, no un índice universal de equidad. RR concede turnos sucesivos a quienes siguen listos, mientras FCFS puede acumular espera detrás de una tarea larga.

Todos los procesos de las cargas finitas probadas terminaron. Eso permite afirmar que no se observó inanición en esas pruebas. Como razonamiento del modelo, con un conjunto finito de procesos y CPU/E/S finitas, ambas políticas progresan hasta terminar. No se evaluaron cargas infinitas ni tareas que nunca cedan o finalicen.

Riesgos y responsabilidades

Riesgo	Tratamiento y alcance
Rendimiento	Un quantum pequeño aumenta despachos y tamaño de trazas. El costo real no se modela; usar cargas didácticas pequeñas.
Estabilidad	Quantum inválido y tiempos negativos se rechazan. Una salida que no puede escribirse genera mensaje legible y código 1.
Integridad de resultados	Entradas nuevas para cada política, hashes del código y recálculo de CSV. Regenerar evidencias después de cambiar código o cargas.
Archivos y seguridad	No se ejecutan cargas externas ni comandos de sistema. El programa escribe en --output y reemplaza sus archivos de resultados: elegir otra carpeta para conservar una ejecución anterior.
Responsabilidad académica	Declarar asistencia de IA, comprender el código y registrar autores y aportes reales. No simular historial del equipo ni aprobación docente.

Limitaciones y mejoras

El modelo no incluye varias CPU, prioridades efectivas, contención de E/S, costos de cambio, caché, memoria ni tiempo real. El campo priority no afecta FCFS ni RR. La cola de eventos y las trazas se mantienen en memoria; no se validó escalabilidad para millones de turnos. No se modifica el planificador del equipo ni se crean procesos del SO.

Como mejoras futuras se propone añadir costo configurable de contexto, cola de dispositivo, gráficos de Gantt y cargas definidas por archivo con validación. Estas son propuestas y no funcionalidades que se afirme haber implementado.

Conclusiones

La versión revisada compara las mismas cargas sin contaminar entradas, representa bloqueo real dentro de la simulación y guarda evidencias reproducibles. Los resultados muestran el compromiso entre respuesta, finalización y frecuencia de cambios. La elección depende de la carga y de la métrica; las simplificaciones impiden trasladar directamente las cifras a un sistema operativo real.

Reproducción y responsabilidad del equipo

Cómo reproducir

Extraer el ZIP, abrir una terminal en la carpeta que contiene main.py y ejecutar: python main.py --output evidencias_nuevas. Ejecutar después: python -m unittest -v. No se necesitan paquetes de pip para el simulador. Se recomienda Python 3.10 o posterior; la versión efectivamente probada fue 3.12.14.

Abrir resumen.csv para comparar políticas; procesos.csv para comprobar cada proceso; cpu.csv para seguir los turnos; estados.csv para observar bloqueo y reactivación. Los archivos incluidos constituyen evidencias de la revisión. Si se modifica scenarios.py o el código, generar una carpeta nueva y actualizar el informe para mantener coherencia.

Correcciones frente al avance recibido

La revisión del ZIP original confirmó que sus tres escenarios se ejecutaban y calculaban las métricas esperadas. Se corrigieron quantum sin validación, división entre cero en reporte vacío, CPU restante incoherente en FCFS y resultados erróneos al reutilizar procesos en RR. Se añadieron validaciones, E/S simulada, trazas, exportación, pruebas y documentación. REVISION.md detalla los hallazgos y las evidencias originales.

Aportes y pendientes de acreditación

Se utilizó asistencia de IA mediante Codex para revisión, corrección, ampliación, pruebas y documentación. APORTES.md identifica esta asistencia y contiene el espacio para registrar únicamente integrantes y contribuciones reales. El equipo debe verificar, comprender y poder explicar toda la versión, conforme al apartado 8 de las instrucciones del curso [4].

El equipo está integrado por Esteban Peña y Andrés Gámez. El código, el informe y las evidencias están disponibles en el repositorio privado https://github.com/Jhomnsito/sistemas-operativos. Sigue pendiente completar la trazabilidad de sus aportes con evidencias reales y dar acceso al profesor cuando corresponda. No se pudo acreditar la revisión de semana 8 ni la sustentación individual. El guion propone una distribución de la exposición y no constituye un registro de trabajo ya realizado.

Referencias

[1] Remzi H. Arpaci-Dusseau y Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces, capítulo 7, Scheduling: Introduction. https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf

[2] Los mismos autores. Operating Systems: Three Easy Pieces, capítulo 4, The Abstraction: The Process. https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf

[3] Python Software Foundation. Documentación de Python 3.12, unittest. Referencia para la ejecución de las pruebas automatizadas. https://docs.python.org/3.12/library/unittest.html

[4] Consigna del profesor. Proyecto Final SO 2620.docx. Requisitos, opción 1, entrega y uso de inteligencia artificial. Documento proporcionado para esta revisión. Referencias web consultadas el 2 de noviembre de 2026.